Computer Architecture
Multicycle Machine

Tatpong Katanyukul

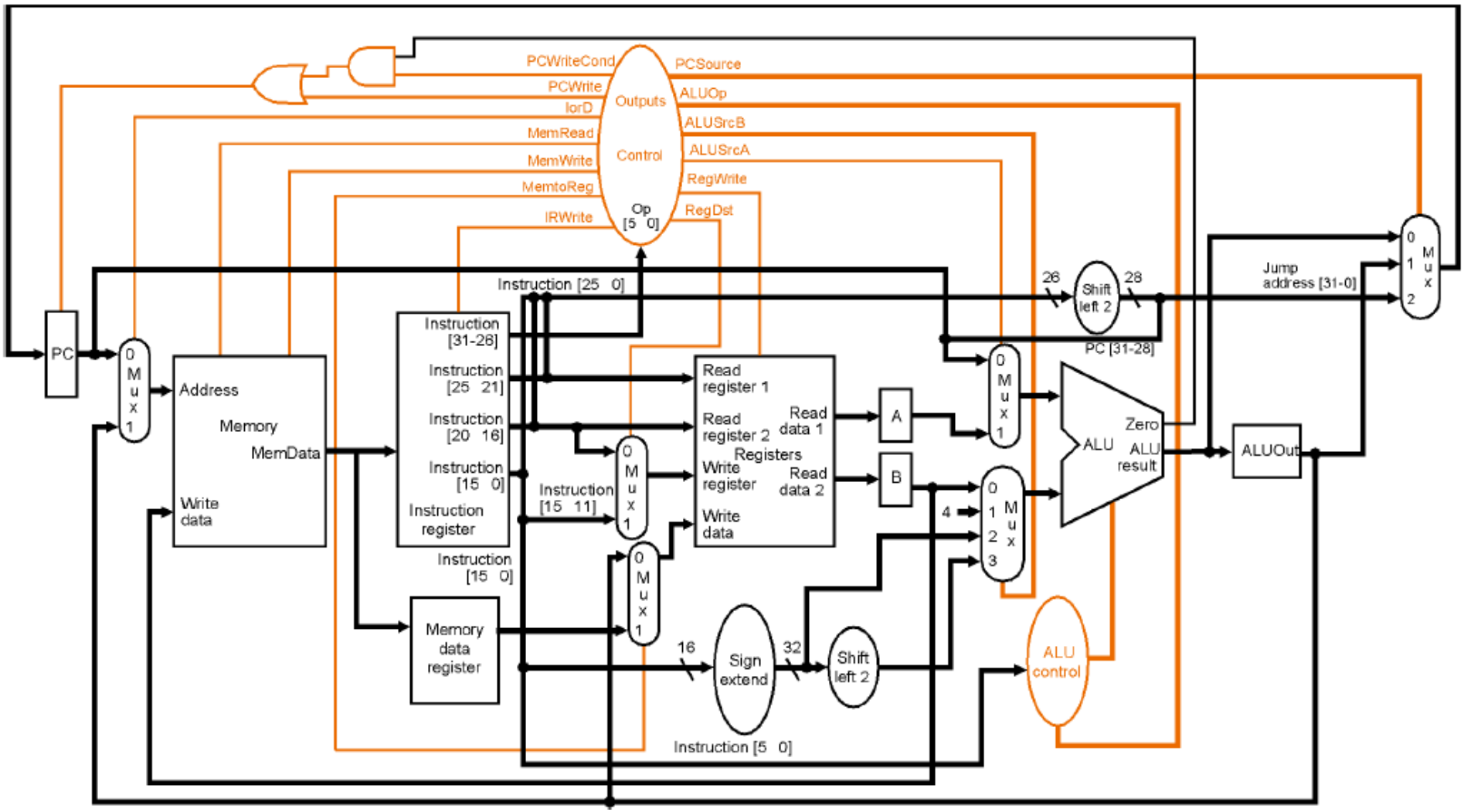
CoE, KKU

24 พฤษภาคม พ.ศ. 2561

Instructions

Instruction	Format	m/c code					
		6	5	5	5	5	6
add	add rd, rs, rt	0	rs	rt	rd	0	0x20
subtract	sub rd, rs, rt	0	rs	rt	rd	0	0x22
and	and rd, rs, rt	0	rs	rt	rd	0	0x24
or	or rd, rs, rt	0	rs	rt	rd	0	0x25
set on less than	slt rd, rs, rt	0	rs	rt	rd	0	0xA
load word	lw rs, rt, offset	0x23	rs	rt	offset		
store word	sw rs, rt, offset	0x2B	rs	rt	offset		
branch on equal	beq rs, rt, offset	4	rs	rt	offset		
jump	j target	2	target				

Multicycle Machine's Full Datapath



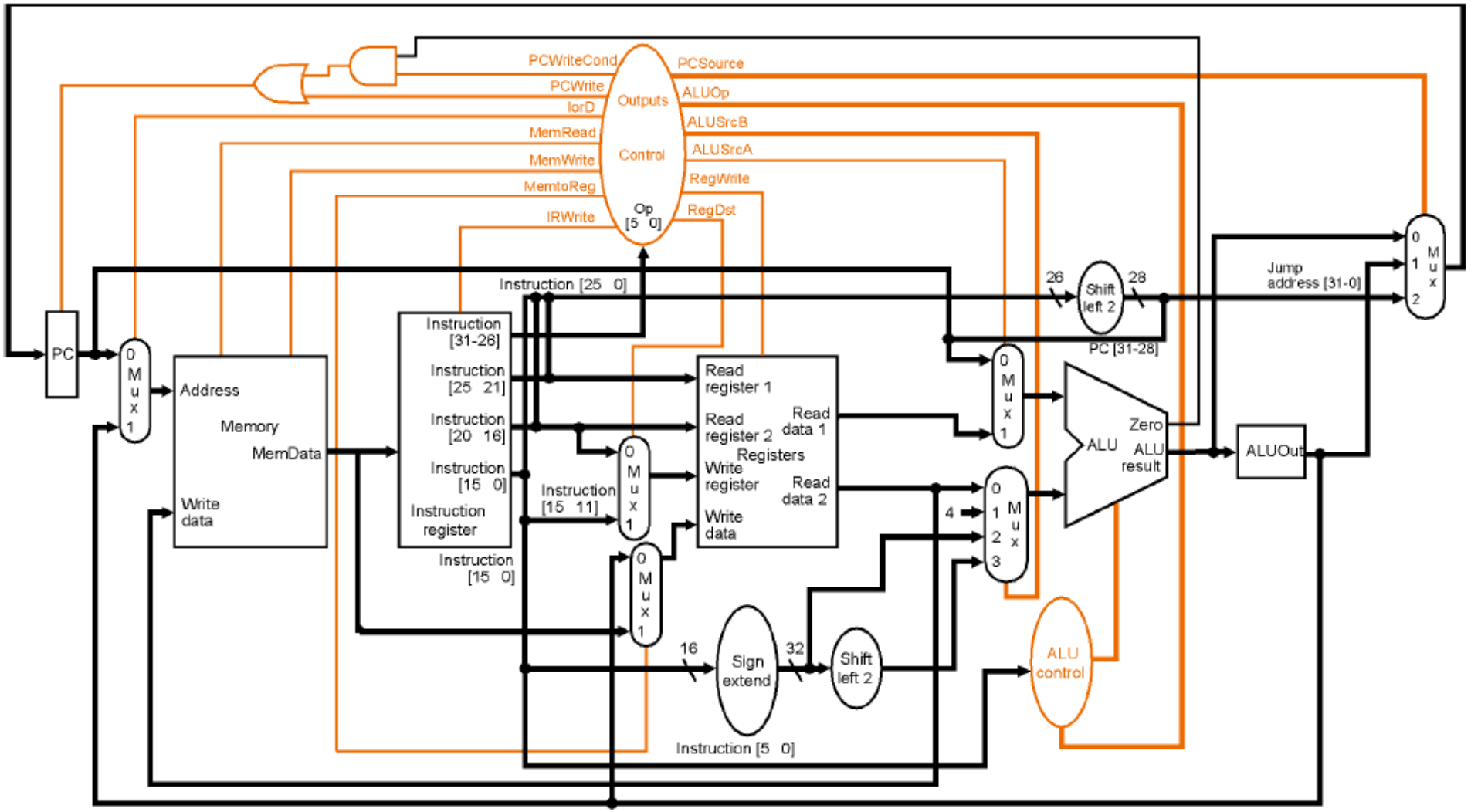
Five-Stage Control

Step	R-type	Load	Store	Branch	Jump
1	IR <= Memory[PC] PC <= PC + 4 IorD = 0, MemRead, IRWrite, ALUSrcA = 0, ALUSrcB = 01, ALUOp =00, PCSource = 00, PCWrite				
2	A <= Reg[IR[25:21]] B <= Reg[IR[20:16]] ALUOut <= PC + (sign-extend (IR[15:0]) << 2) ALUSrcA = 0, ALUSrcB = 11, ALUOp = 00				
3	ALUOut <= A op B ALUSrcA = 1, ALUSrcB = 00, ALUOp = 10	ALUOut <= A + sign-extend(IR[15:0]) ALUSrcA = 1, ALUSrcB = 10, ALUOp = 00		if(A == B) PC <= ALUOut ALUSrcA = 1, ALUSrcB = 00, ALUOp = 01, PCSource = 01, PCWriteCond,	PC <= {PC[31:28], (IR[25:0],00)} PCSource = 10, PCWrite
4	Reg[IR[15:11]] <= ALUOut RegDst = 1, MemtoReg = 0, RegWrite	MDR <= Memory[ALUOut] IorD = 1 MemRead	Memory[ALUOut] <= B IorD = 1 MemWrite		
5		Reg[IR[20:16]] <= MDR RegDst = 0 MemtoReg = 1 RegWrite			

Main Control

- Main control is a sequential logic circuit.
- $\rightarrow$ needs state register
- Input: opcode (6 bits), state in (3 bits)
- Output: state out (3 bits), PCWriteCond, PCWrite, PCSource (2 bits), MemRead, MemWrite, IorD, IRWrite, ALUop (2 bits), ALUSrcA, ALUSrcB (2 bits), RegWrite, RegDst, MemtoReg

Simpler Version: Datapath



Simpler Version: Control Signals

Step	Jump: j	Branch: beq	R-type	Load: lw	Store: sw
0	*IR <= Mem[PC]; PC <= PC + 4* IorD=0, MemRead, IRWrite, ALUSrcA=0, ALUSrcB=01, ALUOp=00, PCSource=00, PCWrite				
1	*PC <= {PC[31:28], IR[25:00],00}* PCSource=10, PCWrite	*ALUOut <= PC + (signext(IR[15:0])<<2)* ALUSrcA=0, ALUSrcB=11, ALUOp=00	*ALUOut <= Reg[IR[25:21]] op Reg[IR[20:16]]* ALUSrcA=1, ALUSrcB=00, ALUOp=10	*ALUOut <= Reg[IR[25:21]] + signext(IR[15:0])* ALUSrcA=1, ALUSrcB=10, ALUOp=00	
2		*if(A==B) PC <= ALUOut* ALUSrcA=1, ALUSrcB=00, ALUOp=01, PCSource=01, PCWriteCond	*Reg[IR[15:11]]<= ALUOut* RegDst=1, MemtoReg=0, RegWrite	*Reg[IR[20:16]]<=Mem[ALUOut]* IorD=1, MemRead, RegDst=0, MemtoReg=1, RegWrite	*Mem[ALUOut]<= Reg[IR[20:16]]* IorD=1, MemWrite

Instructions

Instruction	Format	m/c code					
		6	5	5	5	5	6
add	add rd, rs, rt	0	rs	rt	rd	0	0x20
subtract	sub rd, rs, rt	0	rs	rt	rd	0	0x22
AND	and rd, rs, rt	0	rs	rt	rd	0	0x24
OR	or rd, rs, rt	0	rs	rt	rd	0	0x25
shift left	sll rd, rt, shamt	0	0	rt	rd	shamt	0x00
shift right	srl rd, rt, shamt	0	0	rt	rd	shamt	0x02
set on less than	slt rd, rs, rt	0	rs	rt	rd	0	0x0A
load word	lw rt, imm(rs)	0x23	rs	rt	imm		
store word	sw rt, imm(rs)	0x2b	rs	rt	imm		
branch on equal	beq rs, rt, braddr	0x04	rs	rt	braddr		
jump	j target	0x02	target				
load upper imm	lui rt, imm	0x0F	0	rt	imm		
add imm	addi rs, rt, imm	0x08	rs	rt	imm		
AND imm	andi rs, rt, imm	0x0C	rs	rt	imm		
OR imm	ori rs, rt, imm	0x0D	rs	rt	imm		

Exceptions and Interrupts (1)

Options

- Interrupt vector handler (multiple handler addresses)
- Cause register

MIPS

- MIPS uses Cause register
- Control registers: Status (CR#12), Cause (CR#13), and EPC (CR#14)
- Instructions: mfc0, mtc0, and eret

Exceptions and Interrupts (2)

Our design (inspired by MIPS, but definitely not MIPS!)

- Interrupt vector handler
- 1 interrupt and 1 exception
  - Exception handler at 0x80001000
  - Interrupt handler at 0x80002000
- Control registers: EPC and Status
  - EPC $\leftarrow$ next-instruction address for interrupt
  - EPC $\leftarrow$ address of instruction causing exception
  - Status $\leftarrow$ simplest design using only one bit (Status[0]: enable/disable)

    No nested exception/interrupt

Exceptions and Interrupts: Mechanism (1)

Exception:

- EPC $\leftarrow$ (PC)
- Status $\leftarrow$ 0x00000000
- PC $\leftarrow$ 0x80001000

Interrupt:

- EPC $\leftarrow$ (PC+4)
- Status $\leftarrow$ 0x00000000
- PC $\leftarrow$ 0x80002000

Exceptions and Interrupts: Mechanism (2)

Exception/Interrupt related instructions

mnemonic	opcode	5 bits	5 bits	5 bits	5 bits	6 bits	Hex
mfc0	010000	00000	rt	rd	00000	000000	0x40XXXX00
mtc0	010000	00100	rt	rd	00000	000000	0x40XXXX00
eret	010000	10000	00000	00000	00000	011000	0x42000018

- mfc0: GR[rt] $\leftarrow$ CR[rd]
- mft0: GR[rt] $\rightarrow$ CR[rd]
- eret:
  - PC $\leftarrow$ EPC
- Notice eret does not automatically enable exception/interrupt.
  $\Rightarrow$ to re-enable interrupt, it must be done explicitly

Exceptions and Interrupts: Handler Example

E.g., interrupt handler

```
# At address 0x80002000
lui $s1, IO_DATA_ADDR_HI        # pre-specified I/O address
ori $s1, $s1, IO_DATA_ADDR_LO
lui $s2, READ_DATA_ADDR_HI      # designated memory address
ori $s2, $s1, READ_DATA_ADDR_LO
lw $t0, 0($s1)     # read from I/O data address (memory-maped I/O)
sw $t0, 0($s2)     # write to memory
# Handler program is done.
# Re-enable exception/interrupt
mfc0 $k0, $Status
ori $k0, $k0, 1
mtc0, $k0, $Status
eret
```

Exceptions and Interrupts: Datapath

Closing note

- อ่าน P&H5e, บทที่ 4.
- อ่าน บทที่ 5 เอกสารประกอบรายวิชา 188 322 สถาปัตยกรรมคอมพิวเตอร์, COA notes (April 25, 2014).

